

TP 3 : Pretty Good Privacy (PGP)

ISC_125 – Sécurité des systèmes d'information et cryptographie

Ludovico Grabau et Valentin Acevedo

4 mai 2021

Tables des matières

De quoi s'agit-il.....	2
Démarche, résultats et réponses	2
1. Introduction à PGP	2
2. Installation de GnuPG.....	4
3. Installation des clés GPG.....	5
4. Exportation de la clé publique et envoi de la clé aux correspondants	8
5. Importation de la clé publique depuis ses correspondants pour stockage interne	9
6. Envoi et réception de message cryptés	9
7. Création de la signature.....	11
8. Vérification de la signature.....	12
Autres questions	13
Conclusion	14
Sources	14

De quoi s'agit-il

L'objectif de ce TP est d'étudier le protocole PGP (Pretty Good Privacy) développé par Philip R. Zimmermann en 1991. Il s'agit également de comprendre comment le fonctionnement d'algorithmes asymétriques pour le chiffrement et la signature tels que RSA et Diffie Hellman en utilisant PGP.

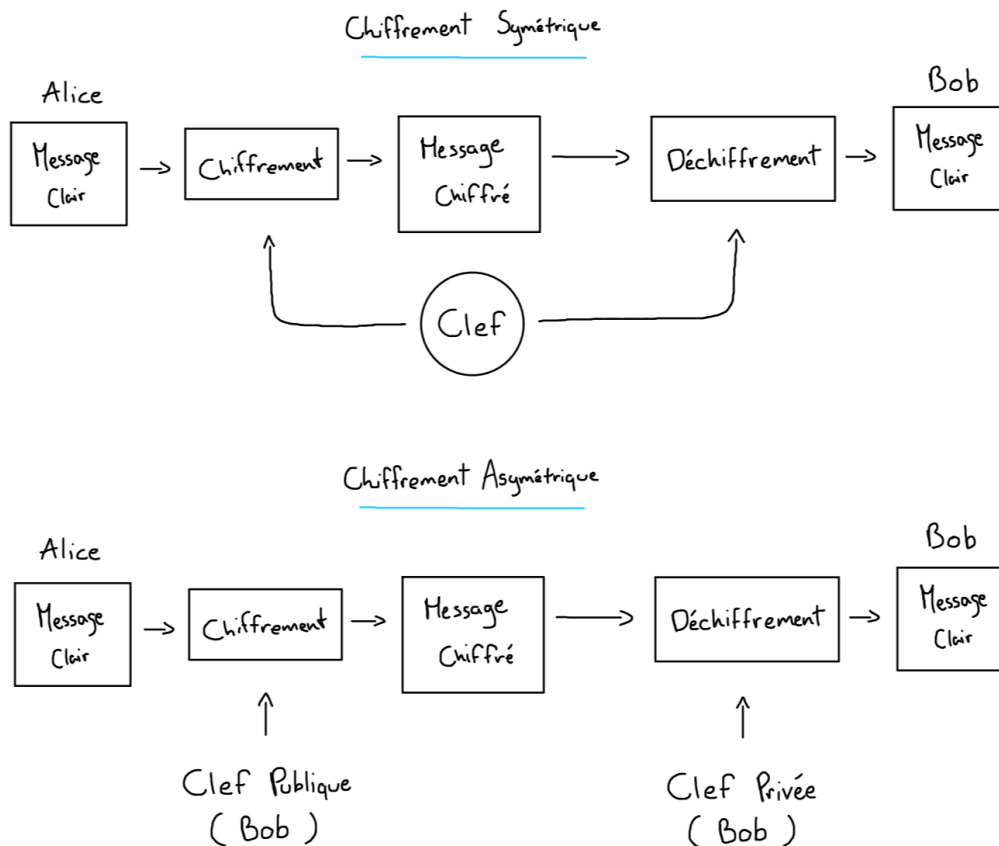
Démarche, résultats et réponses

1. Introduction à PGP

PGP est un programme payant populaire beaucoup utilisé pour crypter et décrypter les courriers électroniques, pour authentifier la source des messages par signature numérique et stocker des fichiers cryptés. OpenPGP est la version libre fondée sur PGP proposée par Philip R. Zimmermann en 1997 à l'IETF¹. Il est reconnu comme standard pour le chiffrement de courriers électroniques. GPG (GNU Privacy Guard) est gratuit et a été développé par Werner Koch en 1999. Il est interopérable avec OpenPgp et PGP. Différentes distributions gratuites existent qu'il s'agisse de distribution Linux ou Windows. Par exemple, Gpg4win2 est une distribution de GPG sous Windows. Dans le cas de ce TP¹, nous allons travailler avec GnuPG pour Linux CentOS 73. Chacune de ces distributions propose sa propre interface graphique.

a) Schéma de fonctionnement (étape de recherche personnelle)

Le chiffrement asymétrique fonctionne à l'aide de deux clefs que forge le destinataire. Celui-ci, dans notre cas Bob, envoie la clef dite publique à Alice pour qu'elle puisse chiffrer un message qu'elle souhaite envoyer à Bob. Lui à une clef privée qu'il utilisera pour déchiffrer le message chiffré d'Alice.



Le concept de certification est essentiel au fonctionnement du chiffrement asymétrique. Grâce à lui nous pouvons nous assurer que les personnes recevant la clef publique sont de confiance, et éviter qu'ils aient leur identité usurpée. Ces certificats sont donc signés par des entités de confiance pour tenir trace de l'intégrité des différents possesseurs de clef publique. Un certificat remplace la fonction que prenait uniquement la clef publique car cette clef fait dorénavant partie du certificat, sa signature est construite à l'aide de la clef privée et il comporte des informations identifiant la personne en question. Trois éléments constituent la force de ces certificats, ils sont chiffrés, ils sont envoyés directement au particulier désiré et ils détiennent un « tampon » d'une autorité de certification.

2. Installation de GnuPG

1. Vérifiez que la dernière version GnuPG de CentOS est préinstallée, sinon installez-la. Quelle(s) commande(s) avez-vous utilisé pour vérifier et installer ?

→ `sudo apt install gnupg`

→ `gpg --version`

2. Lancez la commande sur le manuel GPG et sur d'aide en ligne de GPG

→ `man gnupg`

→ `man gpg`

<https://gnupg.org/documentation/manpage.html>

<https://gnupg.org/documentation/manuals/gnupg.pdf>

<http://irtfweb.ifa.hawaii.edu/~lockhart/gpg/>

3. Après avoir utilisé ces commandes, listez les algorithmes qui peuvent être utilisés en précisant leur utilité/fonction.

<code>-s, --sign</code>	faire une signature
<code>--clear-sign</code>	faire une signature en texte clair
<code>-b, --detach-sign</code>	faire une signature détachée
<code>-e, --encrypt</code>	chiffrer les données
<code>-c, --symmetric</code>	chiffrement symétrique seulement
<code>-d, --decrypt</code>	déchiffrer les données (défaut)
<code>--verify</code>	vérifier une signature
<code>-k, --list-keys</code>	afficher les clefs
<code>--list-signatures</code>	afficher les clefs et les signatures
<code>--check-signatures</code>	afficher et vérifier les signatures de clefs
<code>--fingerprint</code>	afficher les clefs et les empreintes
<code>-K, --list-secret-keys</code>	afficher les clefs secrètes
<code>--generate-key</code>	générer une nouvelle paire de clefs
<code>--quick-generate-key</code>	générer rapidement une nouvelle paire de clefs

<code>--quick-add-uid</code>	quickly add a new user-id
<code>--quick-revoke-uid</code>	quickly revoke a user-id
<code>--quick-set-expire</code>	quickly set a new expiration date
<code>--full-generate-key</code>	générer une paire de clefs complètes
<code>--generate-revocation</code>	générer un certificat de révocation
<code>--delete-keys</code>	supprimer les clefs du porte-clefs public
<code>--delete-secret-keys</code>	supprimer les clefs du porte-clefs secret
<code>--quick-sign-key</code>	signer rapidement une clef
<code>--quick-lsign-key</code>	signer rapidement une clef localement
<code>--sign-key</code>	signer une clef
<code>--lsign-key</code>	signer une clef localement
<code>--edit-key</code>	signer ou éditer une clef
<code>--change-passphrase</code>	modifier une phrase secrète
<code>--export</code>	exporter les clefs
<code>--send-keys</code>	exporter les clefs vers un serveur de clefs
<code>--receive-keys</code>	importer les clefs d'un serveur de clefs
<code>--search-keys</code>	chercher les clefs avec un serveur de clefs
<code>--refresh-keys</code>	mettre à jour les clefs depuis un serveur
<code>--import</code>	importer ou fusionner les clefs
<code>--card-status</code>	afficher l'état de la carte
<code>--edit-card</code>	modifier les données d'une carte
<code>--change-pin</code>	modifier le code personnel d'une carte
<code>--update-trustdb</code>	mettre la base de confiance à jour
<code>--print-md</code>	indiquer les fonctions de hachage
<code>--server</code>	exécuter en mode serveur
<code>--tofu-policy VALUE</code>	set the TOFU policy for a key

...

3. Installation des clés GPG

4. Lancez la commande pour générer une paire de clés avec les options par défaut

→ `gpg --gen-key` Génère une nouvelle paire de clefs

5. Quelle information sur les clés demande-t-on de spécifier à la création des clés ?

On nous demande un "Nom réel".

6. Quelle est la seconde information à fournir ? A quoi sert cette information ?

On nous demande une "Adresse électronique"

7. Quelle est la troisième information à fournir ? A quoi sert cette information ?

On nous demande d'entrer une "Phrase secrète" servant de mot de passe. (1234)

8. Quelles sont les informations renvoyées après la création de votre paire de clés ?
Quel est l'identifiant de votre clé publique ?

gpg: clef 30FA76062627D1EB marquée de confiance ultime.

gpg: répertoire « /home/icestorm/.gnupg/openpgp-revocs.d » créé

gpg: revocation certificate stored as '/home/icestorm/.gnupg/openpgp-revocs.d/F3BC9F8E1A92675C32EE75A530FA76062627D1EB.rev'

les clefs publique et secrète ont été créées et signées.

→ clef publique : F3BC9F8E1A92675C32EE75A530FA76062627D1EB

→ pub rsa3072 2021-05-11 [SC] [expire : 2023-05-11]

F3BC9F8E1A92675C32EE75A530FA76062627D1EB

```
icestorm@iceBreaker:~$ gpg -k
/home/icestorm/.gnupg/pubring.kbx
-----
pub   rsa3072 2021-05-11 [SC] [expire : 2023-05-11]
      F3BC9F8E1A92675C32EE75A530FA76062627D1EB
uid           [  ultime ] Zboubelito <ludovico.grabau@etu.hesge.ch>
sub   rsa3072 2021-05-11 [E] [expire : 2023-05-11]
```

9. Que sont les champs pub, uid et sub ? Pourquoi une empreinte de clé est-elle également générée ? Que pouvez-vous dire sur cette empreinte ?

pub : correspond à la clef publique

uid : les informations sur l'utilisateur (nom réel, adresse électronique)

sub : correspond à la clef privée

L'empreinte sert à l'utilisateur pour qu'il puisse s'identifier.

Elle informe l'utilisateur des informations qui lui sont à sa disposition.

Cette empreinte nous servira à authentifier les divers utilisateurs.

10. A quoi sert un certificat de révocation ? Quand faut-il de préférence le générer ?

Quand la clef a été compromise, remplacée ou qu'elle n'est plus utilisée.

11. Lancez la commande pour la révocation de vos clés.

→ `gpg --generate-revocation` générer un certificat de révocation

12. Lancez la commande listant les clés. Où se trouvent-elles effectivement ?

→ `gpg --list-keys` afficher les clefs

→ `home/icestorm/.gnupg/pubring.kbx`

4. Exportation de la clé publique et envoi de la clé aux correspondants

1. Sous quel type de format en particulier la clé publique peut-elle être exportée ?
Pourquoi est-ce pratique ?

Sous un type de format en binaire. Ce n'est pas spécialement pratique, ce type de format convient pour plusieurs cas mais il peut présenter des problèmes si on désire l'envoyer par courriel. Pour remédier à cela il faut un type de format appelé ASCII-armored à l'aide de l'option « --armor ».

2. Lancez la commande d'exportation de la clé publique.

→ `gpg --export -a "Zboubelito">public.key`

3. Quel est le fichier contenant votre clé publique ? Que contient également ce fichier ?

C'est le fichier "public.key". On dirait qu'il contient un hash sous forme de block, donc le fichier est rempli de caractères divers et variés.

```
icestorm@iceBreaker:~$ cat public.key
-----BEGIN PGP PUBLIC KEY BLOCK-----

mQGNBGCalaUBDADKewIE7dB0pmF5idB7gBH3svdngqq7boudKU/ppE+bKeLeBBbk
0139F1gIyJU7Riy0QXw/FT84AQ2vJJpovYEvU5Sgial8rUVpJ2JBAYtu1nhh/NBv
loHdtRsQzhV/KKxZSaM4+pA//NU1E/TakQao5sJGuJ9pNyXv00gmCUAM/sKm8ty2
STxAWtLwmKu+Fhu9/pryE21rNM2QeTMqQidW5x6QyzZaUzGfTjGnLDLES+p8h0ow
6RMQwUYFZvHapF0b6zp9ttVW9EK2MqM6jNthH3Bj+E65/UMkl5sQC+AovJZISko
JBD8ChefAzLX42IVNWuLG7uAiSvAOMZZKgmt44BT7E8E3kf487rk4GuV6LB3Y2df
sApagATV2Yyl21HT9w3cKvE0c3grYgHTq0TZi1jEtfc5KL8kH3etrNgAa80VTuIZ
p0F7hpt/8H7evLzgkpxyodJSh/9n0ZmcAn/V9qxPp3pXBLJXd4oAg2b3jDZk4pQH
Q/y7UGgZ3u359g8AEQEAAAbQpWmJvdWJlbG0byA8bHVkb3ZpY28uZ3JhYmF1QGV0
dS5oZXNnZS5jaD6JAdQEEwEKAD4WIQTzvJ+OGpJnXDLudaUw+nYGJiFR6wUCYJqV
pQIbAwJABJnAUAULCQgHAgYVCgkICwIEFgIDAQIeAQIeAAKACRAw+nYGJiFR60EW
DADGrmU6hy9wC/K0Xw/DHCQ37gkLT/L/loNwmTRtBDIFGL24nxBfJN9PVCcQLgrS
LsfDHeQvHAR3t0FH6C9fCs9v4jNGeR6/7W5eVfDQcg2kpdQIOmJXrwiq8YwJ2BB
XRed/L4MAWzeA5FrCz/XMIq+CwuIQ5DK+vqgSJS5+wEaY0WvUB6FTA5JaWYBNTGmQ
aYpY0em1IMyhJ5NaCht5rrkst5B0i4gRTC9SGWP2KLFunWezNdrZzqxQP8fWfapi
BkoH2L7UJCeurHfnujvzws9S4pZ0k51q0tiD8Z1Lwm7RWB5GJlwm0J8sBU3t0dZM
JgjfJcPk9yISK0uWq54JclYvYbULe7MzU3pVW3DuFmFeJQz+c+8P/J3Fds7Qr4sU
XIctaDyvfqP4EerGms+RAG8w0kXXvoennjdHLCnww7RKtQGPqRZn9TP4u5Iay9DE
TrVhK/QLHqK9N0SdUmK0IwyVxiJAZhDoevE+F4yRltnLxSmNYHvqWby4y50UVZL
yTW5AY0EYJqVpQEMAM2QSN7g74Ryu57jYGCe6iTnzEazEJ1ti9mm7/Sgl6i4ZwY
ndZLbmSHIEdiYXDrH0PoTD7urSpLguzPr//LCfvqWjyqgxxIdKWyI72G1x3bChE7
Y7aLTor4+wSS19U5wCreHkBDIqBi76q03FzZghgUNBRP+G28ovDLxuRkEMPdgokb
K6iY+WL/YtP2kzHsCpis/2ijBA/08LH905biuJpu70ft7dV/t0TqtKPVJgJK83te
04BRry4FeAcv3+UipuAz4wFU/KDC6YU+aYrqhXGKzEmK0CbG/oJ5ROUoq4Nbsk+z
vI5j5iaQ0sg16IrTtdSegWS9ioa2k1X0dNLRLRz0TxgD+5gxl0gNap0wpkobYPb
/fQvYo5ytMipYhRKlsuQTc5hNMxiDYSsUq8td83WVoQ87wlyU0qL9MM6H6Hop1b1
y7L48xvFNrMVY+YxPZvq88sFwuup62BnVIC9MUM7B7/3Cwp5VNtRAfv40uKg14U+
xgE2dxuK64100fC2swARAQABiQG8BBgBCGAmFiEE87yfhjqSZ1wy7nWLMPP2BiYn
0esFAMCalauCgWwFCQPCZWAACGkQMPP2BiYn0es29Qv/aC2BEHxriUUpfe4IvDT
b/JoaMqKZReVunS82kaGds7YmkjyylRS6e72wx2LGFsXKvv79Asu0GR08aIukXNV
q1iXFYg4z7naCTzPN9VVMUZ1bI27fj2wb8jQvboIshkxZRs/QKX6kIthYQwpGe0o
1AVoNqmJ/aUA2MRP1udoyT3/YfWMMa6DZC1ydvP7L4nIJMKUGo1/8vV1dNTNM8It
VLoCJSsRbv/4ngyXKq4tbFvapKQKPTJvHhpik77Ekbqm750FMY/mAebDV62a6wM
Gr865/2Vq8U1IZUrFtAI7MzCoQ9mo/ej7fQvNU6x5i0JKtG42z1tR8P0Y0Eb2QG/
R8Emp81ZzMAoJ2L9qVqoCXDkzGjK/58cAIwvJGpz07wyzI3g2kuT+NP5Jw+YBMs
bah+k40PUfi1vUBHZR8ae56GCBFs18JagtI06DUKvUahwDjgtuflZfVew5qw8Veh
9/SPa9kCWQF5yGeT/ezb+xFBiKcYSNjgBZ7/thG00C9
=7Amv
-----END PGP PUBLIC KEY BLOCK-----
```

4. Quelles sont les étapes et les commandes effectuées ?

On exporte notre clé publique. Les commandes ci-dessus.

5. Quelle autre méthode auriez-vous pu utiliser pour exporter votre clé auprès de vos collègues ?

On peut utiliser l'option suivante, malgré qu'elle ne cible pas mes collègues :

→ `--send-keys` exporter les clefs vers un serveur de clefs

On peut aussi l'exporter dans un type de format précis, comme indiqué dans la question 4.1.

Finalement on peut modifier la commande pour employer une autre méthode qui est :

→ `gpg --output myFile.gpg --export [email]`

5. Importation de la clé publique depuis ses correspondants pour stockage interne

1. Lancez la commande d'importation de la clé publique.

→ `gpg --import publicKeyOfFriendName.key`

```
icestorm@iceBreaker:~/Téléchargements$ gpg --import dado_public_key.key
gpg: clef 4BA46FF70FCA1F2F : clef publique « Damian Boquete <damian.boquete-cost
a@etu.hesge.ch> » importée
gpg: Quantité totale traitée : 1
gpg: importées : 1
```

6. Envoi et réception de message cryptés

1. Choisissez/Créez un fichier texte à sauvegarder.

→ `nano text`

2. Lancez la commande pour chiffrer votre fichier.

→ `gpg -e -u "Zboubelito" -r "FriendHostname" textToMyFriendName(.txt)`

3. Quelle Clé avez-vous utilisée ?

Nous avons utilisé la clé publique.

4. Quel message apparaît ? Pourquoi ?

```
icestorm@iceBreaker:~/.gnupg$ gpg -e -u "Zboubelito" -r "Damian Boquete" text
gpg: DAD19E8DE25B1D20 : aucune assurance que la clé appartienne vraiment à l'utilisateur nommé.

sub rsa3072/DAD19E8DE25B1D20 2021-05-11 Damian Boquete <damian.boquete-costa@etu.hesge.ch>
  Empreinte clé princip. : B1A1 462D 2B5A 52B2 A8D5 DC55 4BA4 6FF7 0FCA 1F2F
    Empreinte de sous-clé : 533E B74C D4A2 7D2D 96CD E18F DAD1 9E8D E25B 1D20

La clé n'appartient PAS forcément à la personne nommée
dans l'identité. Si vous savez *vraiment* ce que vous
faites, vous pouvez répondre oui à la prochaine question.

Faut-il quand même utiliser cette clé ? (o/N) o
icestorm@iceBreaker:~/.gnupg$
```

5. Comment pouvez-vous vous assurer que la clé provient bien du bon utilisateur ?

En se mettant en contact avec la personne nommée est en faisant bien attention que c'est le cas.

6. Quelle autre information est donnée dans ce message ? Pourquoi ?

L'empreinte de la clé principale, l'empreinte de la sous-clé, des messages de préventions en lien à l'assurance que la clé correspond réellement à l'utilisateur nommé.

7. Sous Quel Format Votre Fichier Crypté Apparaît-il? Que pouvez-vous dire de sa taille ?

Sous un format .gpg. Le fichier textToMyFriendName.gpg fait 558 octets.

8. Envoyer le fichier en tant que pièce jointe à vos deux correspondants.

Fait !

9. Lancez la commande pour déchiffrer ces fichiers.

→ `gpg --decrypt textFromMyFriendName.gpg > "textFromMyFriendName"`

10. Quelle Clé avez-vous utilisée ?

J'utilise ma clé privée.

11. Quelle autre étape/information a été nécessaire au moment du déchiffrement ?

L'entrée de la passphrase prédéfinie.

12. Expliquer son utilité.

On l'utilise pour avoir accès à la clé privée.

13. Si vous envoyez un message crypté à un correspondant qui ne vous a pas partagé sa clé publique, que se passe-t-il ?

On ne pourra pas chiffrer l'information, car l'utilisation d'une clé publique est essentielle pour cette action.

7. Création de la signature

1. Lancez la commande pour la signature unique.

→ `gpg -s textToMyFriendNameSigned(.txt)`

2. Sous quel format est la signature ?

La signature est sous format binaire.

3. Lancez la commande contenant la signature et le fichier original.

→ `gpg --clearsign -o textSigned(.txt) textToMyFriendNameSigned`

4. Quel format renvoie cette commande ?

Il renvoie un format texte.

5. Quelle clé utilisez-vous pour signer votre message ?

J'utilise ma clé privée.

6. Éditez Le Fichier Signé et examinez les algorithmes qui y figurent ? Pourquoi un algorithme de hachage est-il indiqué ?

Le SHA512 est utilisé comme algorithme. Il est employé pour pouvoir vérifier l'intégrité du fichier.

8. Vérification de la signature

1. Lancez la commande pour vérifier la signature d'un message de vos correspondants.

→ `gpg --verify textFromFriendNameSigned.gpg`

2. Quelle clé utilisez-vous pour vérifier la signature d'un message de vos correspondants ?

Nous utilisons la clé publique appartenant au correspondant en question.

3. Que se passe-t-il si votre correspondant modifie le message après l'avoir signé et vous l'envoie ?

On se retrouve à avoir un hash corrompu qui n'est plus valable ce qui fait que la signature ne pourra plus être vérifiée. Cependant, le message sera toujours accessible et nous pourrons le lire.

Autres questions

1. Ai-je besoin de choisir le mot de passe pour crypter PGP ?

Oui nous avons besoin de choisir un mot.

2. Si j'envoie un mail crypté à un destinataire qui n'utilise pas OpenPGP, que se passe-t-il ? Pouvez-vous utiliser votre paire de clés sur un autre ordinateur ?

Oui nous pourrons toujours faire fonctionner les fichiers créés à l'aide de OpenPGP par un destinataire n'utilisant pas ce dernier. L'important c'est que les clefs soient valables et dans un format qui répond aux exigences par défaut, ce qui est fait automatiquement. Il nous faut juste s'assurer que ces fichiers soient faits localement.

3. Quel est l'intérêt de PGP si vous utilisez une connexion sécurisée type SSL ou SSH ?

Cela nous permettra de rajouter une couche de sécurité, ce qui peut être nécessaire dans certaines applications.

4. Quel est l'intérêt de PGP par rapport d'autres solution cryptage (MCrypt, TrueCrypt, cryptage Windows)

Dans le cas de PGP nous utilisons des méthodes de cryptage dites asymétriques et nous avons la possibilité de chiffrer divers messages, puis de les envoyer.

Conclusion

En conclusion, nous avons réussi à encrypter un message envoyé par courriel grâce au protocole Pretty Good Privacy (PGP) à l'aide du logiciel GnuPG. Nous avons généré des clefs asymétriques, que nous avons partagé aux futurs destinataires de notre message, nous avons encrytés nos fichiers à l'aide de notre clef, nous avons envoyé par courriel notre message chiffré, nous avons attendu que le destinataire confirme avoir reçu et déchiffrer notre fichier, nous avons créé une signature et nous l'avons vérifié. Grâce à cela nous avons complété notre envoi de fichiers chiffrés par courriel, pouvant être déchiffré, lu et vérifié par notre destinataire.

Sources

https://fr.wikipedia.org/wiki/Certificat_électronique#cite_note-6

<https://culture-informatique.net/c-est-quoi-un-certificat-numerique/>

<https://culture-informatique.net/cest-quoi-un-certificat-ssl/>

<https://blog.emsisoft.com/fr/27699/rancongiciels-chiffrement/>

https://moodle.utc.fr/pluginfile.php/16777/mod_resource/content/0/SupportIntroSecu/co/CoursSecurite_32.html

<https://gnupg.org/gph/fr/manual.html#AEN68>